HuGraph: An Economical Cloud-Native RDF Store

Bo Zhai
Hunan University
Changsha, China
zhaibo@hnu.edu.cn

Peng Peng
Hunan University
Changsha, China
hnu16pp@hnu.edu.cn

M. Tamer Özsu
University of Waterloo
Waterloo, Canada
tamer.ozsu@uwaterloo.ca

Lei Zou
Peking University
Beijing, China
zoulei@pku.edu.cn

Qian Lei
Hunan University
Changsha, China
l12345678@hnu.edu.cn

ABSTRACT

The growing scale of RDF data has driven interest in cloud-native solutions that offer elastic scalability and cost efficiency. However, existing RDF stores are predominantly designed for integrated compute-storage architectures and fail to exploit the economic and operational benefits of modern cloud platforms—particularly storage disaggregation. In this paper, we present *HuGraph*, a cloud-native RDF store that minimizes monetary cost while delivering acceptable query performance under time constraints. HuGraph adopts a disaggregated architecture: at the storage layer, it organizes the RDF graph by grouping all triples that share the same property into a dedicated subgraph. Each such subgraph is stored as an independent object in low-cost object storage (e.g., Amazon S3), augmented with a novel *Range-Aware Offset Index* (RAOX) to enable efficient range-based data retrieval. At the computation layer, it models both the cost and the execution time of cloud storage operations. It formulates SPARQL query execution as a cost-minimization problem under time constraints—an NP-hard optimization task for which we propose an effective branch-and-bound algorithm. Experimental results show that HuGraph significantly reduces operational costs compared to traditional RDF stores, without sacrificing query performance in practical workloads.

PVLDB Reference Format:
Bo Zhai, Peng Peng, M. Tamer Özsu, Lei Zou, and Qian Lei. HuGraph: An Economical Cloud-Native RDF Store. PVLDB, 14(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The *R*esource *D*escription *F*ramework (RDF) is a standard model for data exchange on the web. The fundamental unit of RDF is the *triple* (subject, property, object), which models a statement about a resource: the subject denotes the resource, the property denotes the relationship or attribute, and the object denotes the value or related resource. An RDF dataset is a collection of RDF triples and can be viewed as a graph. In this graph, subjects and objects serve as vertices, while triples form the edges, with properties acting as the labels on these edges. SPARQL (SPARQL Protocol and RDF Query Language) is the standard language used for querying and manipulating RDF graphs. A SPARQL query consists of *triple patterns*, similar to RDF triples, but with variables in place of subjects, properties, or objects. A SPARQL query can also be viewed as a query graph, and answering a SPARQL query involves finding subgraphs within the RDF graph that are homomorphic to the query graph [34].

As in their relational counterparts, managing RDF data on the cloud has become an attractive solution as data size grows. Cloud environments provide more cost-effective and elastic computing compared to traditional on-premise deployments. Cloud platforms allow for dynamic scaling of resources, and the large cloud vendors have major economies of scale relative to on-premise setups.

However, building a cloud-native RDF store presents unique challenges. Modern cloud architectures typically decouple storage and compute, allowing each layer to be scaled and billed independently. This architectural shift necessitates a fundamental rethinking of both RDF storage layout and query processing strategies. Moreover, while cloud computing is often promoted for its economic efficiency and operational flexibility, this very advantage elevates cost[1] to be a first-class design criterion along with performance. In many real-world applications, extreme query performance is secondary to predictable and minimized operational expenditure; thus, economic efficiency is a critical objective in system design. This gives rise to two core challenges:

(1) **Storage Design Challenge.** Designing an efficient and cost-effective RDF graph storage and indexing scheme that fully leverages cloud-native storage services.

[1] In this paper, unless otherwise noted, "cost" refers to monetary cost.

(2) Query Processing Challenge. Developing a SPARQL query processing engine that operates in a storage-disaggregated architecture and aligns with the I/O patterns and cost structure of cloud storage, with the goal of minimizing cost while delivering acceptable query performance.

Existing systems fall short in addressing these requirements. Traditional RDF stores such as Jena [16], RDF4J [2], Virtuoso [13], and gStore [33] are built on an integrated compute-storage architecture. When deployed directly in the cloud, they require fixed resource allocation, making it difficult to scale dynamically in response to workload fluctuations. This often results in underutilized resources and unnecessary expenditure. Conversely, modern cloud-native relational DBMSs such as Presto [25], Snowflake [12], Redshift Spectrum [5], and Apache Iceberg [1] typically adopt storage disaggregation that separates computation and storage, allowing each to be independently managed and connected via a network. However, there are not many cloud-native DBMSs that are optimized for RDF data or SPARQL query semantics, leaving a gap in the landscape of RDF data management on the cloud.

In this paper, we propose a cloud-native RDF store, HuGraph, designed to minimize costs while meeting the performance requirements of user queries. Similar to most cloud-native DBMSs, we adopt the disaggregated architecture that divides computation and storage into separate layers connected through the network.

At the storage layer, to address the storage design challenge, we use the remote shared object storage (e.g., Amazon's Simple Storage Service (S3) or Alibaba's Object Storage Service (OSS)) to store the RDF graphs, since cheap remote shared object storage provides order of magnitude cost savings over remote block stores like EBS, without significant performance disadvantages in mixed workloads [26]. We vertically divide the RDF graph into subgraphs where all triples with the same property are put into the same subgraph, and then each subgraph is organized as an independent file in the remote shared object storage. To enhance data access efficiency, we introduce an index structure called the Range-Aware PSO set Index (RAOX). This index records the subject and object values of triples associated with a given property, along with their corresponding byte offsets within the respective file in the remote shared object storage. By leveraging this index, HuGraph supports six access methods that leverage the Range GET and Range SELECT operators natively offered by cloud object storage. We analyze the cost and latency characteristics of these methods and propose a cost model that captures both execution time and economic cost.

At the computation layer, when a SPARQL query is submitted with a specified time constraint, the problem of query processing can be formulated as an optimization problem aimed at minimizing cost while satisfying the time requirement. We prove that this problem is NP-hard and propose a branch-and-bound algorithm to determine the optimal query execution strategy.

Contributions. We make the following contributions in the paper.

- We propose a new RDF store, HuGraph, designed specifically for cloud computing environments, utilizing a disaggregated storage-computation architecture.
- At the storage layer, we leverage remote shared object storage to reduce costs and introduce the Range-Aware PSO set Index (RAOX) to accelerate data access. We analyze the access methods and develop a model that captures both execution time and economic cost.
- At the computation layer, we formalize query processing and design a branch-and-bound algorithm that yields optimal execution strategies.
- We evaluate HuGraph on synthetic and real-world RDF graphs across multiple cloud platforms. Experimental results show that our system significantly lowers cost compared to existing RDF stores, without meaningful performance degradation on the tested workloads.

2 BACKGROUND

2.1 Preliminaries

In RDF, resources on the web are uniquely identified by Internationalized Resource Identifiers (IRIs). Using this concept, we define the RDF graph as follows, following the definitions from [21].

Definition 2.1. (RDF Terms, Triple and Graph). The set of RDF terms consists of the set of IRIs I, the set of blank nodes B, and the set of literals L. A triple $(s, p, o) \in (I \cup B) \times I \times (I \cup B \cup L)$ is called an RDF triple, where s is the subject, p is the property and o is the object. A set of triples is called an RDF graph.

An RDF dataset can be represented as a graph where subjects and objects are vertices and triples are labeled edges.

Meanwhile, SPARQL is a structured query language over RDF where the primary building block is the basic graph pattern (BGP). In this study, we focus on BGP queries as they are foundational to SPARQL, and we consider techniques for handling them.

Definition 2.2. (Basic Graph Pattern) Let V be a set of variables; variables bind to RDF terms from $I \cup B \cup L$. A triple in $(I \cup V) \times (I \cup V) \times (I \cup V \cup L)$ is called triple pattern. We omit blank nodes from triple patterns for ease of exposition. Let P be the set of all triple patterns. Then, a basic graph pattern (BGP) is a set $Q \subseteq P$. A set of queries is $Q \subseteq 2^P$.

Example 2.1. Figures 1 and 2 illustrate an example RDF graph and a SPARQL BGP, respectively. These examples are extracted from the well-known DBpedia RDF graph [17] and represent factual information about several English philosophers.

For a BGP, the set of triples from an RDF graph that satisfy all its triple patterns is referred to as its matches. The formal definition is provided as follows.

Definition 2.3. (BGP Match) Here, we denote the partial function $\mu : V \to I \cup B \cup L$ as a mapping μ from V to $I \cup B \cup L$. Abusing notation, for a triple pattern t, we denote by $\mu(t)$ the triple obtained by replacing the variables in t according to μ.

The set of matches for Q is denoted as $[\![Q]\!]_G$.

A SPARQL can also be seen as a query graph. We assume that query Q is a weakly connected directed graph; otherwise, each connected component of Q is considered separately.

2.2 Cloud Environment

2.2.1 Pricing of Cloud Services. Since the primary design goal of HuGraph as a cloud-native RDF store is to minimize costs while

Table 1: AWS service prices

Service	Speci cation	Price (US$)
S3 Storage	On demand	0.023/Month/GB
EBS Storage	On demand	0.08/month/GB
S3 PUT	On demand	0.005/1000 requests
S3 GET	On demand	0.0004/1000 requests
S3 SELECT	Scan	0.002/GB
	Return	0.0007/GB
EC2	2C2G	0.0208/hour
	2C4G	0.0464/hour
	2C8G	0.096/hour
	4C16G	0.192/hour
	8C32G	0.384/hour
	32C128G	1.536/hour
	64C256G	3.072/hour
Neptune Computing	2C4G (Standard)	0.098/hour
Neptune Storage	On demand	0.1/month/GB
Neptune Storage IO	On demand	0.2/million IOs
VPC	On demand	0.01/endpoint/hours

Figure 1: Example RDF graph

```
SELECT ?G ?~ ?I WHERE{
  ?G influencedBy ?~.
  ?~ deathDate 1951-04-29.
  ?I mainInterest 'Ethics'.
  ?G birthPlace 'London'.
  ?~ influencedBy ?I}
```

Figure 2: Example basic graph pattern query

meeting performance requirements, it is essential rst to consider the pricing of the cloud services used in its implementation. Thus, we begin by listing the relevant service prices, using Amazon Web Services (AWS) as a representative, since our implementation uses AWS services. However, HuGraph is designed to be deployable across di erent cloud providers that o er scalable object storage and virtualized compute services comparable to AWS. To demonstrate this exibility, we validate HuGraph on Alibaba Cloud in our experiments. Table 1 provides an overview of AWS service pricing.

Amazon S3.Amazon's Simple Storage Service (S3) is a highly scalable and durable object storage service for storing and retrieving any amount of data from anywhere on the web. Within S3, data is stored as objects, typically in formats like CSV or Parquet. One of the key advantages of Amazon S3 is its low cost. As shown in Table 1, the storage cost is only $0.023 per GB per month, making it signi cantly lower than other storage services.

Data in S3 can be accessed using several methods, including the GET operator and the SELECT operator. Further, both GET and SELECT operators can invoke the Range functionality to form Range GET or Range SELECT operators, which allows users to download or scan speci c byte ranges of an object, avoiding the need to retrieve and scan the entire le.

The PUT and GET operator as core methods of S3 for storing and retrieving objects in the cloud with obvious semantics (GET retrieves an entire le). These operations are very cheap. Regardless of the amount of data involved, the GET operator costs just $0.0004 per 1,000 requests, making the associated costs negligible in most practical applications.

The SELECT is a paid service that enhances the capabilities of querying data in S3. By scanning the data within les and returning only the data that matches the speci ed query criteria to the computing node, it introduces limited support for SQL query syntax. Currently, SELECT supports select, projection, and aggregation queries. Its performance is notably better than standard GET operator. However, SELECT operator incurs additional costs, which are based on the amount of data scanned and returned. As shown in Table 1, the price for scanning data is $0.002 per GB, and for returned data, the price is $0.0007 per GB.

Amazon EC2.Amazon EC2 (Elastic Compute Cloud) is a core service in AWS that provides scalable virtual servers, called instances, to run applications in the cloud. EC2 provides a wide range of instance types tailored to di erent workloads, each with varying pricing models. Table 1 lists the instance types and their corresponding costs used in our experiments.

Amazon EBS.Amazon EBS (Elastic Block Store) provides persistent block storage for use with Amazon EC2 instances. It o ers low-latency, high-performance storage and is ideal for any application requiring raw storage access. As shown in Table 1, the storage cost is $0.023 per GB per month. To access data, the EBS volume is often attached to an EC2 instance and mounted to a directory. EBS volumes act as block storage, and after mounting, they can be used like local hard drives. Accessing data stored in EBS through an EC2 instance does not involve any additional costs.

Neptune.Neptune [3] is a fully managed graph database service o ered by AWS that can support RDF graphs. Neptune should be deployed within a Virtual Private Cloud (VPC), which can provide network isolation. While Neptune simpli es the process of building and managing graph database services, it comes with a high price. As shown in Table 1, the total cost of using Neptune consists of four components: computing, storage, storage I/O, and the cost of an EC2 instance to access the Neptune service. These combined factors result in a relatively high overall cost.